

Preguntas de análisis

1. ¿Cuál fue el problema principal: el avión, el software o ambos? Justifique.

R/

El problema fue de ambos , teniendo en cuenta que por la negligencia del fabricante (Boeing) se tuvo que crear un software para cubrir un error de diseño físico y el propio software presentó fallas críticas por falta tanto de conocimiento previo del software , como por pruebas de calidad hacia el mismo software ya que el software dependía únicamente de un solo sensor sin tener en cuenta previamente pruebas de estrés .

2. ¿Qué decisiones de la empresa aumentaron el riesgo del proyecto?

R/

Fueron 3 la principales decisiones las cuales aumentaron el riesgo:

-Ocultamiento de información por reducción de costos: No informaron de la existencia del software para no pagar capacitaciones a las aerolíneas y a los pilotos .

-Eliminación de redundancia: diseñaron el software para depender de un solo sensor.

-Priorizaron los tiempos de entrega y las acciones financieras: aceleraron el desarrollo para competir directamente con Airbus , recortando los tiempos de testing e ignorando advertencias técnicas.

3. ¿Qué actividades de aseguramiento de la calidad (QA) considera que no se realizaron o fueron insuficientes?

R/

-Evaluación y gestión de riesgos:NO se clasificó el MCAS como un sistema crítico para la seguridad.

-Auditoría de requisitos: NO se validó el requisito de diseño teniendo en cuenta el hardware/software.

-Revisión de pares: NO se tuvieron en cuenta alertas de ingenieros y pilotos de prueba que expresaron dudas sobre el comportamiento del sistema.

4. ¿Qué documentos técnicos debieron existir antes de poner el software en producción?

R/

-Matriz de Trazabilidad de Requisitos (RTM): Para asegurar que cada fallo potencial tuviera una mitigación de software probada.

-Documentación del Arquitectura del Sistema: Detallando la dependencia de sensores y algoritmos.

-Manual de Usuario: Con procedimientos claros para la desactivación del software en caso de mal funcionamiento.

-Plan de Pruebas y Reportes de Ejecución de QA: Firmados y autorizados por auditores independientes.

5. ¿Qué tipo de pruebas considera que hacían falta?

R/

-Pruebas de Tolerancia a Fallos: Simular deliberadamente la falla o corrupción de datos del sensor AoA para verificar la respuesta del software.

-Pruebas de Integración Hardware-Software: Validar la interacción entre los sensores físicos, la computadora de vuelo y la interfaz humana.

-Pruebas de Usabilidad: Evaluar cómo reaccionan pilotos reales en simuladores bajo estrés cuando el sistema enviaba alertas falsas.

-Pruebas de Límite: Evaluar el comportamiento del MCAS cuando recibía lecturas extremas del sensor.

6. ¿Quién debe responder cuando un fallo crítico llega a producción?

R/

-Gerencia de Proyecto: Por recortar presupuesto y crear una cultura donde se ignoran las alertas de calidad.

-Líderes de QA y Arquitectos de Software: Por aprobar el despliegue a producción sabiendo que existía un punto único de falla.

7. ¿Qué costos produjo la falta de calidad (económicos, legales, humanos y reputacionales)?

R/

-Económicos: Pérdidas multimillonarias, cancelación de pedidos, cancelaciones de vuelos.

-Legales: Demandas colectivas, multas del gobierno de EE.UU. e investigaciones penales.

-Humanos: La trágica pérdida de 346 vidas humanas en los accidentes.

-Reputacionales: Destrucción total de la confianza histórica en el nombre de Boeing como líder mundial en aviación.

8. Si usted fuera QA Lead, ¿qué acciones habría exigido antes de aprobar el despliegue?

R/

-Exigir redundancia obligatoria de datos (comparación entre 2 o más sensores) antes de permitir que el software actúe sobre los mandos.

-Hacer no negociable la inclusión del software en los manuales de capacitación y simuladores de vuelo.

-Exigir la implementación de un mecanismo de anulación manual instantáneo y accesible para los pilotos.

-Bloquear el despliegue firmando un reporte de riesgo crítico no mitigado.

9. Relacione el caso con su proyecto ADSO. ¿Qué riesgos similares podrían presentarse?

-Omitir validaciones críticas: Lanzar funcionalidades o APIs a producción sin validar entradas de datos de los usuarios, lo que puede congelar la app o corromper la base de datos.

-Falta de pruebas con entornos reales: Probar el sistema únicamente en un entorno de desarrollo ideal y no someterlo a pruebas de estrés y redes lentas.

-Documentación inexistente: Desarrollar código sin documentar la lógica o las APIs, dejando a los futuros desarrolladores o usuarios sin saber cómo responder ante un error del sistema.

10. Escriba tres conclusiones sobre la importancia de planificar la calidad desde el inicio del desarrollo.

R/

La calidad no es un parche final, es un atributo de diseño.

Prevenir es inmensamente más barato que corregir.

El QA protege a las personas: El rol de QA no es solo buscar errores en el código, sino garantizar que el producto sea seguro, usable y ético.